



Concert Table Reservation System for Restaurant

Group Members

SE 101

Chetsada Winaikoson	6772012221
Waris Natirutthakorn	6872081621
Natcha Thawinpipat	6870083521
Watayut Aiamanan	6970267821

This document is an integral component of the subject

2110521 Software Architecture

Department of Software Engineering, Faculty of Engineering, Chulalongkorn University



1. Scope of this deliverable

This document presents the first version of the microservice architecture for the Concert Table Reservation System (CTRS). It contains the Service–Operations–Collaborators table and the architecture diagram of the system, together with the reasoning that ties them to the three business use cases of Deliverable #1.

The architecture covers the three business use cases end to end, including their alternative and exceptional flows:

- UC-01 Reserve a Specific Table — the customer browses concert rounds, holds a table for 15 minutes, completes the profile, accepts the terms, pays the full table fee through the Payment Gateway and receives a QR e-ticket by LINE.
- UC-02 Check In Using Digital QR Ticket — Front Staff scans the e-ticket at the door, the booking reference is verified against the check-in window, the entry is confirmed and the table becomes occupied; bookings not checked in by the end of the grace period become no-shows.
- UC-03 Create Concert Event — the Manager creates a concert round on the venue zone map, prices the table types, validates and publishes it, which makes the round bookable in UC-01 and defines the check-in window used in UC-02.
- UC-04 Create Venue Zone Map — the Manager defines and names venue zones, places tables with unique table numbers, assigns table types and seating capacities, uploads seat-view photographs, validates and activates the map, which makes it selectable in UC-03 and provides the floor plan used in UC-01 and UC-02 for rounds that use it.

Maintaining the venue zone map is not itself one of the three business use cases. It appears in the architecture because a round cannot be created without it, and it is owned by the same service that owns the rounds.



1. Service–Operations–Collaborators

Operations are the business operations exposed by each service. Collaborators are the services or adapters that the service invokes to complete its own operations; an em dash means the service completes its work without calling anyone else.

Service	Operations	Collaborators
Concert Round Service	createOrUpdateZoneMap() setTableType() uploadSeatViewPhoto() createRound() copyRound() saveRoundAsDraft() validateRound() previewRound() publishRound() unpublishRound() editPublishedRound() getUpcomingRounds() getRound() getRoundTables() getRoundPricing() getCheckInWindow()	Table Availability Service initializeRoundTableStatus() Booking Service getConfirmedBookingCount() Media Storage Adapter storeSeatViewPhoto()



Table Availability Service	<code>initializeRoundTableStatus()</code> <code>getRoundTableStatus()</code> <code>holdTable()</code> <code>extendHold()</code> <code>releaseHold()</code> <code>markTableBooked()</code> <code>markTableOccupied()</code> <code>releaseTable()</code> <code>publishTableStatusUpdate()</code>	—
Booking Service	<code>createHeldBooking()</code> <code>setPartySize()</code> <code>calculateTableFee()</code> <code>saveCustomerProfile()</code> <code>acceptBookingTerms()</code> <code>startPayment()</code> <code>confirmBookingPayment()</code> <code>issueETicket()</code> <code>getETicket()</code> <code>cancelBooking()</code> <code>getCustomerBookings()</code> <code>findBookings()</code>	Concert Round Service <code>getRound()</code> <code>getRoundPricing()</code> <code>getCheckInWindow()</code> Table Availability Service <code>holdTable()</code> <code>extendHold()</code> <code>releaseHold()</code> <code>markTableBooked()</code> <code>markTableOccupied()</code> <code>releaseTable()</code> Payment Service



	<code>checkInBooking()</code> <code>recordAdditionalGuests()</code> <code>escalateCheckIn()</code> <code>resolveEscalation()</code> <code>expireUnpaidBookings()</code> <code>markNoShows()</code> <code>getConfirmedBookingCount()</code>	<code>createPaymentRequest()</code> <code>requestRefund()</code> Notification Service <code>sendBookingConfirmation()</code> <code>sendHoldExpiredNotice()</code> <code>sendRefundNotice()</code>
Payment Service	<code>createPaymentRequest()</code> <code>receivePaymentResult()</code> <code>getPaymentStatus()</code> <code>pollPendingPaymentResults()</code> <code>requestRefund()</code> <code>submitTransferSlip()</code> <code>reviewTransferSlip()</code>	Payment Gateway Adapter <code>createCheckoutSession()</code> <code>verifyWebhookSignature()</code> <code>queryPaymentStatus()</code> <code>refundPayment()</code> Media Storage Adapter <code>storeTransferSlip()</code> Booking Service <code>confirmBookingPayment()</code>
Notification Service	<code>sendBookingConfirmation()</code> <code>sendHoldExpiredNotice()</code> <code>sendRefundNotice()</code> <code>sendSlipDecisionNotice()</code>	LINE Messaging Adapter <code>pushLineMessage()</code>



	retryFailedMessages() getDeliveryStatus()	
--	--	--



Architecture diagram

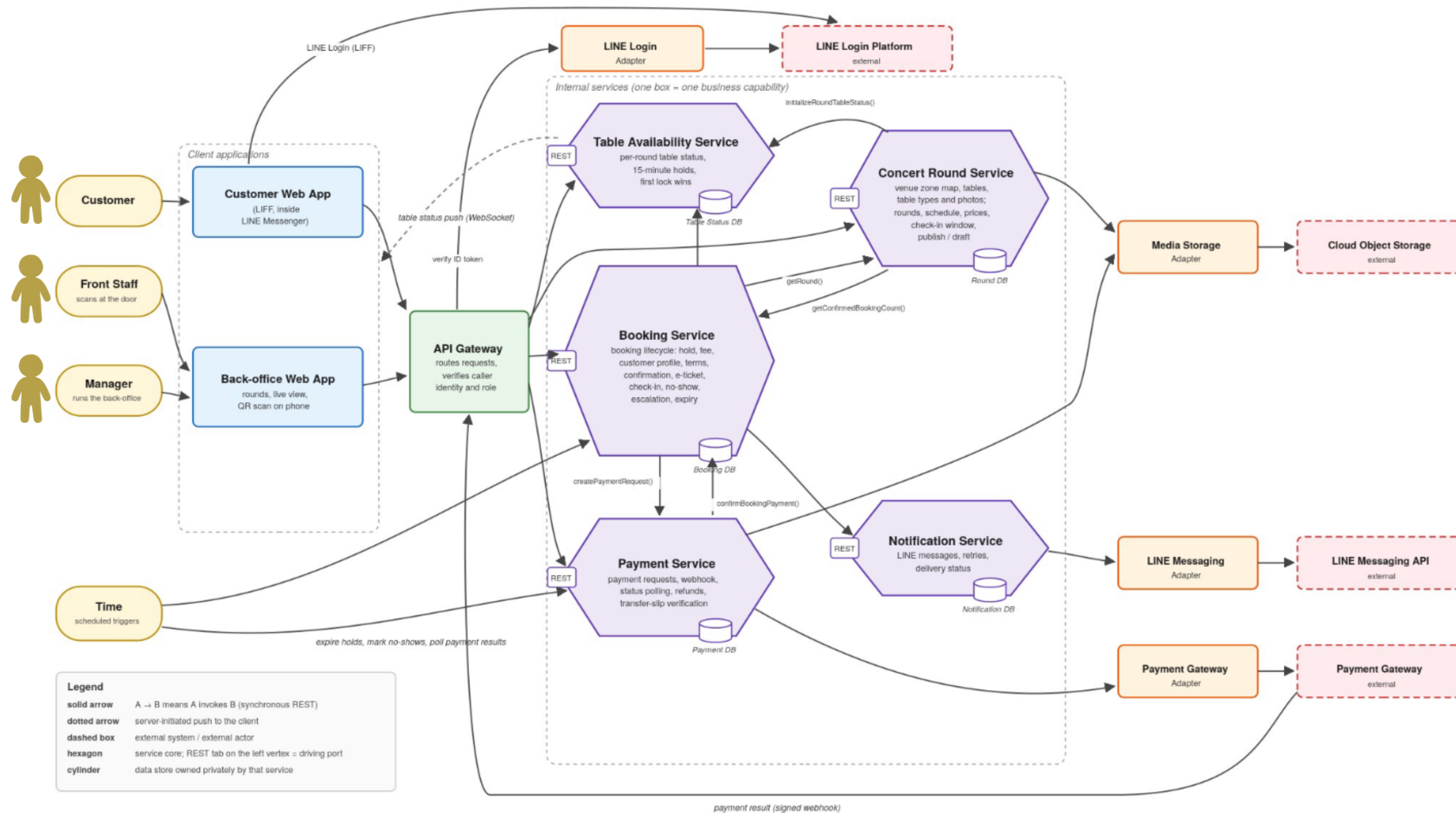


Figure 1 CTRS microservice architecture, version 1, drawn in the ports-and-adapters style: every service is a hexagon whose business logic is reached only through adapters on its edges. An arrow A → B means A invokes B; response paths are not drawn. The hosted checkout page of the Payment Gateway is opened by the customer's browser inside the web app and is therefore not shown as a service call.

